SKYLITE / SKYNET

Framework Technical Manual

For Developers

Version 1.0

March 08, 2026

Table of Contents

This comprehensive manual covers all aspects of SKYLITE and SKYNET framework development, from basic concepts to advanced patterns.

1. Introduction

1.1 What is SKYLITE/SKYNET?

SKYLITE and SKYNET are lightweight web application frameworks designed for C# developers who want to build enterprise-grade web applications with minimal JavaScript and maximum productivity.

• **SKYLITE** - Built on .NET Framework 4.5+ for IIS (Windows-only)

• **SKYNET** - Built on ASP.NET Core / .NET 6+ (Cross-platform: Windows, Linux, macOS)

1.2 Key Philosophy

Less JavaScript, More C#

The framework embraces a server-centric architecture where business logic stays on the server (secure, maintainable) while the client makes lightweight AJAX calls. Developers write 80-90% of their application logic in C#, using JavaScript only for UI interactions.

1.3 Core Benefits

• Minimal JavaScript required - stay in C# for most development

• 30+ pre-built ToolKit controls with professional styling

• Automatic page routing based on class names

• Built-in multi-language support

• Server-centric AJAX pattern for security and maintainability

• Single DLL dependency for easy deployment

2. Architecture Overview

2.1 Core Components

Framework Core (SKYNET)

• IHandler Middleware - ASP.NET Core middleware for request/response handling

• WebPage Class - Base class with event-driven lifecycle

• ToolKit Controls - 30+ pre-built UI components

• ApiRequest/ApiResponse - AJAX communication layer

Framework Core (SKYLITE)

• HttpHandler - IIS request handler

• WebPage Class - Same base class interface as SKYNET

• ToolKit Controls - Same 30+ UI components

• ApiRequest/ApiResponse - Same AJAX pattern

2.2 Request Lifecycle

GET Request (Initial Page Load)

Event Flow:

```
OnLoad → OnInitialized → OnBeforeRender → OnAfterRender
```

1. OnLoad (Framework Automatic)

- Framework looks for MyPage.html template file
- If found, loads content into HtmlDocument.HtmlBodyText
- Creates and formats HtmlDocument (HTML5 standard)

2. OnInitialized (Developer Override) ✓

- Developer builds UI here
- Apply security policy
- Add ToolKit controls

- Configure page layout

3. OnBeforeRender (Developer Override) ✓

- Inject initial JavaScript to HtmlDocument
- Complete HtmlDocument
- Apply content translation (multi-language)

4. OnAfterRender (Developer Override) ✓

- Final chance to modify before sending
- Framework sends HtmlDocument to browser

POST Request (AJAX Call)

Event Flow:

```
OnRequest → [Developer Method] → OnResponse
```

1. OnRequest (Developer Override) ✓

- Framework automatically calls page class and method
- Developer can add pre-processing, validation, security checks

2. OnResponse (Framework Automatic)

- Apply content translation if needed
- Send ApiResponse object to browser as JSON

3. Getting Started

3.1 Prerequisites

For SKYLITE:

• .NET Framework 4.5 or higher

• IIS 7.0 or higher (Windows only)

• SQL Server 2012 or higher (recommended)

• Visual Studio 2017 or higher

For SKYNET:

• .NET 6 or higher (cross-platform)

• IIS, Kestrel, or Docker

• SQL Server 2012 or higher (recommended)

• Visual Studio 2022 or VS Code

3.2 Project Structure

```
YourProject/
├── codes/              # Your business logic (C# classes)
│   ├── CRM/           # Module folders (optional)
│   ├── HR/
│   └── Home.cs        # Your page classes
├── Bin/
│   └── SkyNet.dll     # Framework (single dependency)
├── appConfig/
│   └── application.cfg
├── htmls/             # HTML email templates
├── scripts/           # JavaScript files
│   ├── Home.js
│   └── WebScript.js
├── styles/            # CSS files
│   └── WebStyle.css
├── data/              # Data storage
├── images/            # Static images
├── logs/              # Application logs
├── temp/              # Temporary files
└── Program.cs         # SKYNET: ASP.NET Core entry
    or Web.config      # SKYLITE: IIS configuration
```

3.3 Hello World Example

Create your first page in 3 steps:

Step 1: Create a C# class in codes/ folder

```
// File: codes/HelloWorld.cs
using skylite;
using skylite.ToolKit;

public class HelloWorld : WebPage
{
    public override void OnInitialized()
    {
        // Create a simple label
        var label = new Label("Hello, World!");
        label.Wrap.SetStyle(HtmlStyles.fontSize, "24px");
        label.Wrap.SetStyle(HtmlStyles.color, "#2E75B6");

        // Add to page
        HtmlDoc.HtmlBodyAddOn = label.HtmlText;
    }
}
```

Step 2: Access the page

```
Navigate to: http://localhost/YourProject/HelloWorld
```

Step 3: That's it!

The framework automatically:

• Finds the HelloWorld class

• Calls OnInitialized()

• Renders the HTML

• Sends to browser

4. WebPage Class Reference

The WebPage class is the foundation of all pages in SKYLITE/SKYNET. Every page class must inherit from WebPage.

4.1 Properties

HtmlDocument HtmlDoc

The HTML document object being built. Access to add content, scripts, styles, etc.

Common HtmlDoc properties:

• HtmlBodyAddOn - Your custom HTML content

• HtmlBodyText - Template file content (if loaded)

• InitialScripts - Scripts to execute on page load

4.2 Common Methods

string ParamValue(string key, bool isCookie = false)

Get value from query string or cookie.

string GetDataValue(string key)

Get value from AJAX request data.

Example:

```
public ApiResponse SaveUser()
{
    var userName = GetDataValue("userName");
    var userId = GetDataValue("userId");
    // ... save to database
}
```

5. ToolKit Controls Library

SKYLITE/SKYNET includes 30+ pre-built UI controls that generate clean, styled HTML5 with minimal JavaScript.

5.1 Input Controls

Texts Control

Multi-purpose text input supporting various types:

```
// Text input
var textBox = new Texts("User Name", "userName", TextTypes.text);
textBox.Required = true;

// Password input
var password = new Texts("Password", "pwd", TextTypes.password);

// Email input
var email = new Texts("Email", "email", TextTypes.email);

// Date input
var date = new Texts("Birth Date", "birthDate", TextTypes.date);
```

Supported TextTypes:

text, password, email, number, tel, url, date, month, week, time, datetime-local, color, search

Dropdown Control

```
// Simple dropdown
var dropdown = new Dropdown("Country", "country");
dropdown.AddOption("US", "United States");
dropdown.AddOption("CA", "Canada");

// Database-bound dropdown
var roleDropdown = new Dropdown("Role", "roleId");
roleDropdown.Query = "SELECT RoleId, RoleName FROM XysRole";
```

Other Input Controls

```
// CheckBox
var checkbox = new CheckBox("Active", "isActive");
checkbox.Checked = true;

// Switch (toggle)
var toggle = new Switch("Enabled", "enabled");

// TextArea
var notes = new TextArea("Notes", "notes");
notes.Rows = 5;

// FileUpload
```

```
var upload = new FileUpload("Attachment", "file");

// Button
var btn = new Button("Save", Button.ButtonTypes.Button);
btn.SetAttribute(HtmlEvents.onclick, "SaveData()");
```

5.2 Data Display Controls

SQLGridSection - Auto-Generated Data Grid

The most powerful control - generates a complete data grid with sorting, pagination, and search from a single SQL query:

```
var grid = new SQLGridSection();
grid.Query = "SELECT UserId, UserName, UserEmail, Created FROM Users";
grid.PageSize = 20;
grid.EnableSearch = true;
grid.EnableSort = true;

HtmlDoc.HtmlBodyAddOn = grid.HtmlText;
```

Features automatically included:

- Column sorting (click headers)
- Pagination with page numbers
- Search/filter box
- Responsive layout
- Professional styling

TreeView - Hierarchical Data

```
var tree = new TreeView();
tree.Query = "SELECT OrgId, ParentId, OrgName FROM Organization";
tree.IdColumn = "OrgId";
tree.ParentColumn = "ParentId";
tree.DisplayColumn = "OrgName";
```

6. AJAX Pattern & ApiResponse

SKYLITE/SKYNET uses a simple, powerful AJAX pattern for dynamic updates without page reloads.

6.1 Client-Side: $ApiRequest

```
// JavaScript (in your .js file)
function SaveUser() {
    // Collect data from form
    var data = [
        { key: 'userId', vlu: document.getElementById('userId').value },
        { key: 'userName', vlu: document.getElementById('userName').value },
        { key: 'userEmail', vlu: document.getElementById('userEmail').value }
    ];

    // Call server method
    $ApiRequest('SaveUser', JSON.stringify(data));
}
```

6.2 Server-Side: Method & ApiResponse

```
// C# (in your WebPage class)
public ApiResponse SaveUser()
{
    // Get data from request
    var userId = GetDataValue("userId");
    var userName = GetDataValue("userName");
    var userEmail = GetDataValue("userEmail");

    // Business logic
    if (string.IsNullOrEmpty(userName))
    {
        var errorResponse = new ApiResponse();
        var dialog = new DialogBox("User name is required");
        errorResponse.PopUpWindow(dialog.HtmlText);
        return errorResponse;
    }

    // Save to database
    PutData("UPDATE Users SET UserName=@p0, UserEmail=@p1 WHERE UserId=@p2",
            userName, userEmail, userId);

    // Return success response
    var response = new ApiResponse();
    response.Navigate("UserList");  // Redirect to list page
    return response;
}
```

6.3 ApiResponse Commands

ApiResponse provides many commands to control the browser:

Navigation:

```
response.Navigate("PageName");  // Go to another page
```

DOM Manipulation:

```
response.SetElementContents("divId", "<p>New content</p>");
response.ReplaceElement("divId", grid.HtmlText);
response.SetElementAttribute("btnSave", "disabled", "true");
response.SetElementValue("userName", "John Doe");
```

Dialogs:

```
var dialog = new DialogBox("Success!");
response.PopUpWindow(dialog.HtmlText);
response.PopOff();  // Close dialog
```

JavaScript Execution:

```
response.ExecuteScript("alert('Saved!');");
response.ExecuteScript("RefreshGrid();");
```

Storage & Cookies:

```
response.SetCookie("key", "value");
response.RemoveCookie("key");
response.StoreLocalValue("key", "value");
```

File Operations:

```
response.DownloadFile("/path/to/file.pdf");
```

7. Best Practices

7.1 Project Organization

SKYLITE/SKYNET doesn't enforce a rigid folder structure. Organize your codes/ folder however makes sense for YOUR project.

The One Rule: Unique Class Names ⚠

Since the framework uses assembly-based search to find pages, each page class must have a unique name within your project.

✓ Valid:

```
codes/CRM/CrmCustomers.cs       // Class: CrmCustomers
codes/Sales/SalesCustomers.cs   // Class: SalesCustomers
```

✗ Invalid:

```
codes/CRM/Customers.cs          // Class: Customers ❌
codes/Sales/Customers.cs        // Class: Customers ❌ CONFLICT!
```

7.2 Security Best Practices

Always Use Parameterized Queries

✓ CORRECT:

```
PutData("UPDATE Users SET UserName=@p0 WHERE UserId=@p1", userName, userId);
```

✗ WRONG (SQL Injection Risk):

```
PutData("UPDATE Users SET UserName='" + userName + "' WHERE UserId=" + userId);
```

8. Migrating from SKYLITE to SKYNET

Migrating from SKYLITE (.NET Framework) to SKYNET (ASP.NET Core) is straightforward with near-zero code changes.

8.1 What Stays the Same

• WebPage class - Same base class, same lifecycle

• ToolKit controls - Same controls, same API

• ApiRequest/ApiResponse - Same AJAX pattern

• Folder structure - Same organization

• Business logic - Your C# code stays the same

8.2 What Changes

Configuration:

• SKYLITE: Web.config → SKYNET: appsettings.json

Startup:

• SKYNET requires Program.cs with IHandler middleware registration

8.3 Migration Steps

1. Create new ASP.NET Core project

2. Copy your codes/ folder as-is

3. Copy scripts/, styles/, htmls/, images/ folders

4. Replace SkyLite.dll with SkyNet.dll in Bin/

5. Convert Web.config settings to appsettings.json

6. Add Program.cs with IHandler middleware

7. Test and deploy

Estimated migration time: 1-2 days for typical project

9. Resources & Support

9.1 Official Resources

Demo Site:

https://www.theskylite.com/SkyLiteProject

GitHub Repository:

https://github.com/hkim6000/SkyNetDemo-AspNetCore

9.2 Getting Help

• Review the code examples in the showcase project

• Check GitHub issues for similar problems

• Study the Training_Example_Codes document

Appendix A: Quick Reference

WebPage Lifecycle Events

Event	Override?	Purpose
OnLoad	No	Framework loads template, creates HtmlDocument
OnInitialized	Yes ✓	Build UI, add controls
OnBeforeRender	Yes ✓	Inject scripts, final modifications
OnAfterRender	Yes ✓	Logging, analytics

End of Technical Manual

SKYLITE / SKYNET Framework

For additional support and updates, visit:

https://github.com/hkim6000/SkyNetDemo-AspNetCore